

lab3_linear_regression

June 29, 2025

1 Lab Exercise 3: Linear Regression

Overview

Linear regression is one of the foundational tools in predictive analytics, allowing us to quantify and interpret the relationship between one or more continuous predictors and a continuous outcome. In this laboratory exercise, we will explore both the simplest case—regressing a single feature against miles-per-gallon (mpg)—and the more realistic multivariate setting where multiple engine and vehicle characteristics jointly explain fuel efficiency. Along the way we will examine which features carry the most predictive power, how collinearity affects our estimates, and how simple polynomial basis expansions can capture mild non-linearities in the data.

Objectives

1. **Data preparation & cleaning** – Load the Auto dataset, identify non-numeric entries (in horsepower), and impute any missing values.
2. **Simple linear regression** – Fit seven separate OLS models (mpg vs. each feature). – Report slope, intercept, R^2 and p-value for each. – Visualize the fitted line on a scatter plot.
3. **Multiple linear regression** – Fit a full model using all seven predictors. – Interpret coefficient significance and overall fit (R^2 , F-statistic).
4. **Reduced model selection** – Drop predictors that are not statistically significant in the full model. – Compare adjusted R^2 between the full and reduced models to confirm no loss of explanatory power.
5. **Coefficient comparison** – Contrast univariate slopes with multivariate coefficients in a single plot. – Understand how collinearity inflates or attenuates feature-level effects.
6. **Non-linear basis functions** – Extend the best continuous predictors with 2nd-order polynomial terms. – Evaluate whether adding a quadratic term significantly improves model fit.

1.1 Dependencies:

```
[1]: # Core data handling & stats
import pandas as pd
import numpy as np
from scipy import stats

# Visualization
```

```

import matplotlib.pyplot as plt
import seaborn as sns

# Modeling
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm

# Set a consistent plotting style
sns.set_theme(style="whitegrid")

```

1.2 Data:

1.2.1 Load the Auto dataset:

```

[2]: df_auto = pd.read_csv("Auto.csv")

# Inspect basic structure
print("Dataset shape:", df_auto.shape)
print("\nColumn info:")
print(df_auto.info())

# Peek at the first few rows
display(df_auto.head())

# Check for missing values
print("\nMissing values per column:")
print(df_auto.isnull().sum())

```

Dataset shape: (397, 9)

Column info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 397 entries, 0 to 396

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	mpg	397 non-null	float64
1	cylinders	397 non-null	int64
2	displacement	397 non-null	float64
3	horsepower	397 non-null	object
4	weight	397 non-null	int64
5	acceleration	397 non-null	float64
6	year	397 non-null	int64
7	origin	397 non-null	int64
8	name	397 non-null	object

dtypes: float64(3), int64(4), object(2)

memory usage: 28.0+ KB

None

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	\
0	18.0	8	307.0	130	3504	12.0	70	
1	15.0	8	350.0	165	3693	11.5	70	
2	18.0	8	318.0	150	3436	11.0	70	
3	16.0	8	304.0	150	3433	12.0	70	
4	17.0	8	302.0	140	3449	10.5	70	

	origin	name
0	1	chevrolet chevelle malibu
1	1	buick skylark 320
2	1	plymouth satellite
3	1	amc rebel sst
4	1	ford torino

Missing values per column:

```
mpg          0
cylinders    0
displacement 0
horsepower   0
weight       0
acceleration 0
year         0
origin       0
name         0
dtype: int64
```

1.2.2 Cleanup the Auto dataset:

```
[3]: # Convert 'horsepower' to numeric (coerce parse errors → NaN)
df_auto['horsepower'] = pd.to_numeric(df_auto['horsepower'], errors='coerce')
print(f"> Converted 'horsepower' to numeric. dtype is now_
↳ {df_auto['horsepower'].dtype}")

# How many NaNs appeared?
missing_hp = df_auto['horsepower'].isnull().sum()
print(f"> Missing 'horsepower' entries after conversion: {missing_hp}")

# Impute those NaNs by the column mean
hp_mean = df_auto['horsepower'].mean()
df_auto['horsepower'] = df_auto['horsepower'].fillna(hp_mean)
print(f"> Filled missing 'horsepower' with mean = {hp_mean:.2f}")

# Final check: no more missing values anywhere
print("\nFinal missing values per column:")
print(df_auto.isnull().sum())
```

> Converted 'horsepower' to numeric. dtype is now float64

```
> Missing 'horsepower' entries after conversion: 5
> Filled missing 'horsepower' with mean = 104.47
```

Final missing values per column:

```
mpg          0
cylinders    0
displacement 0
horsepower   0
weight       0
acceleration 0
year         0
origin       0
name         0
dtype: int64
```

1.2.3 Discussion & Observations

1. Dataset shape & types

- We have **397 rows** and **9 columns**.
- Three columns are floating-point (`mpg`, `displacement`, `acceleration`), four are integer (`cylinders`, `weight`, `year`, `origin`), and two are object-typed (`horsepower`, `name`).

2. Preview & missing-value check

- A quick `df.head()` shows that `name` holds the full model string (e.g. “chevrolet chevelle malibu”) which we won’t use for numeric modeling.
- Running `df_auto.isnull().sum()` confirms **zero** missing values in every column *before* cleaning.

3. Horsepower needs special handling

- Although `df_auto.info()` reported no nulls, `horsepower` was typed as `object` because several entries failed to parse as numbers (e.g. “?” or blank).
- After `pd.to_numeric(..., errors='coerce')`, we discovered **5** horsepower values became `NaN`.

4. Imputation choice

- We replaced those 5 missing horsepower values with the **column mean** (104.47).
- This is a simple but reasonable strategy here—horsepower is roughly symmetrically distributed, so mean imputation preserves overall scale without introducing bias.

5. Final schema

- Post-imputation, **no NaNs remain** anywhere.
- Now all descriptive variables (`cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `year`, `origin`) are numeric, and ready for both univariate and multivariate regression.
- We’ll drop the `name` column going forward (it’s a free-text identifier), and treat `origin` as either numeric or dummy-encoded depending on the modeling approach.

Overall, this cleanup leaves us with a complete, all-numeric design matrix: perfect for fitting simple OLS, multiple regression, or even polynomial expansions in the next steps.

1.3 Simple Linear Regression on each feature:

```
[4]: # 1) Define which columns to treat as predictors
descriptive_features = [
    'cylinders',
    'displacement',
    'horsepower',
    'weight',
    'acceleration',
    'year',
    'origin'
]
response = 'mpg'

# 2) Write a helper that fits & plots a simple OLS
import statsmodels.api as sm

def fit_and_plot_simple(df, x_col, y_col):
    X = sm.add_constant(df[x_col])      # add intercept term
    y = df[y_col]
    model = sm.OLS(y, X).fit()

    print(f"\n--- Simple Regression: {y_col} ~ {x_col} ---")
    print(model.summary())

    # plot scatter + fit line
    plt.figure(figsize=(5,4))
    plt.scatter(df[x_col], y, alpha=0.6, s=30, label='data')
    plt.plot(df[x_col], model.fittedvalues, 'r--', lw=2, label='fit')
    plt.xlabel(x_col)
    plt.ylabel(y_col)
    plt.legend()
    plt.tight_layout()
    plt.show()

# 3) Loop through each predictor
for feat in descriptive_features:
    fit_and_plot_simple(df_auto, feat, response)
```

--- Simple Regression: mpg ~ cylinders ---

OLS Regression Results

```
=====
Dep. Variable:          mpg    R-squared:                0.603
Model:                  OLS    Adj. R-squared:           0.602
```

```

Method:                Least Squares    F-statistic:                598.9
Date:                  Sun, 29 Jun 2025  Prob (F-statistic):        3.67e-81
Time:                  08:11:18          Log-Likelihood:            -1196.4
No. Observations:      397              AIC:                      2397.
Df Residuals:          395              BIC:                      2405.
Df Model:               1
Covariance Type:       nonrobust

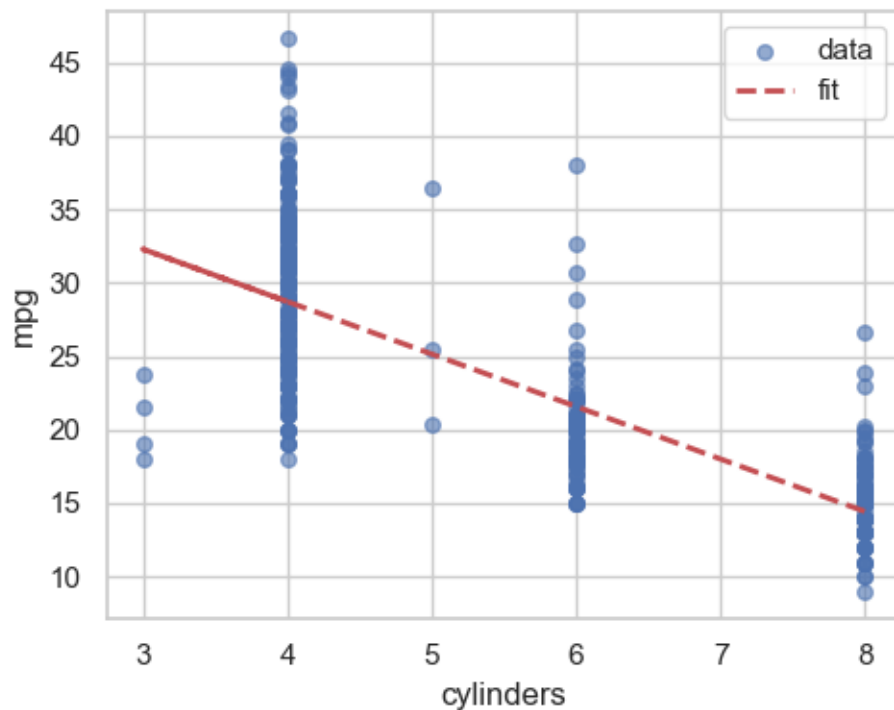
```

	coef	std err	t	P> t	[0.025	0.975]
const	43.0032	0.834	51.563	0.000	41.364	44.643
cylinders	-3.5701	0.146	-24.473	0.000	-3.857	-3.283

Omnibus:	38.686	Durbin-Watson:	1.016
Prob(Omnibus):	0.000	Jarque-Bera (JB):	53.253
Skew:	0.698	Prob(JB):	2.73e-12
Kurtosis:	4.126	Cond. No.	19.8

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



--- Simple Regression: mpg ~ displacement ---

OLS Regression Results

```

=====
Dep. Variable:          mpg      R-squared:                0.647
Model:                  OLS      Adj. R-squared:            0.646
Method:                 Least Squares      F-statistic:          724.4
Date:                  Sun, 29 Jun 2025      Prob (F-statistic):    2.25e-91
Time:                  08:11:18      Log-Likelihood:        -1172.8
No. Observations:      397      AIC:                   2350.
Df Residuals:          395      BIC:                   2358.
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	35.1883	0.493	71.433	0.000	34.220	36.157
displacement	-0.0603	0.002	-26.914	0.000	-0.065	-0.056

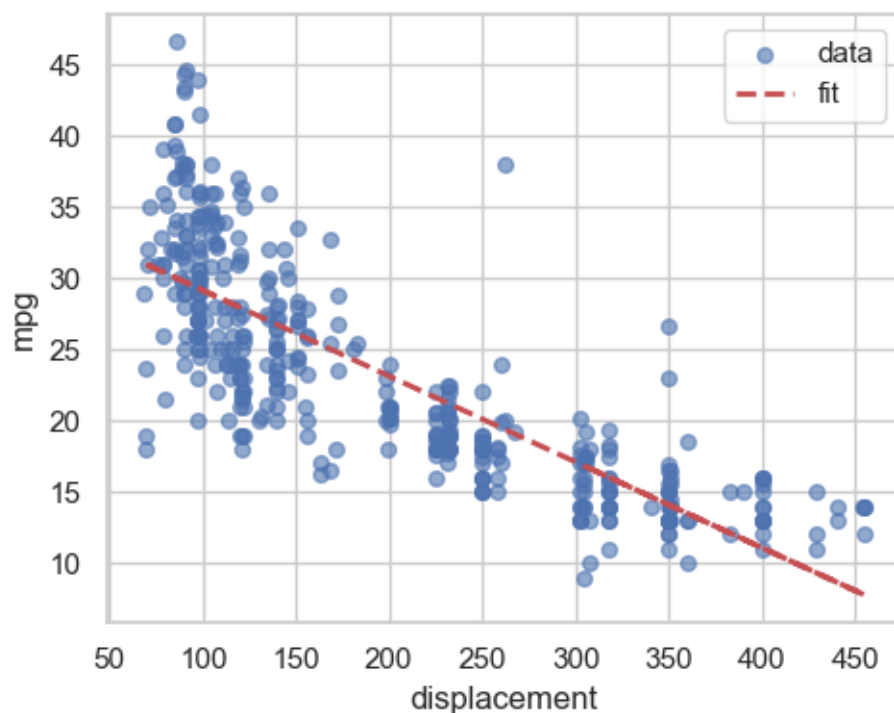
```

=====
Omnibus:                40.862      Durbin-Watson:          0.920
Prob(Omnibus):           0.000      Jarque-Bera (JB):       59.120
Skew:                    0.706      Prob(JB):               1.45e-13
Kurtosis:                4.257      Cond. No.               464.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



--- Simple Regression: mpg ~ horsepower ---

OLS Regression Results

```

=====
Dep. Variable:          mpg      R-squared:                0.595
Model:                  OLS      Adj. R-squared:           0.594
Method:                 Least Squares      F-statistic:        580.6
Date:                  Sun, 29 Jun 2025     Prob (F-statistic):    1.45e-79
Time:                  08:11:19     Log-Likelihood:       -1200.1
No. Observations:      397      AIC:                2404.
Df Residuals:          395      BIC:                2412.
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	40.0058	0.729	54.903	0.000	38.573	41.438
horsepower	-0.1578	0.007	-24.096	0.000	-0.171	-0.145

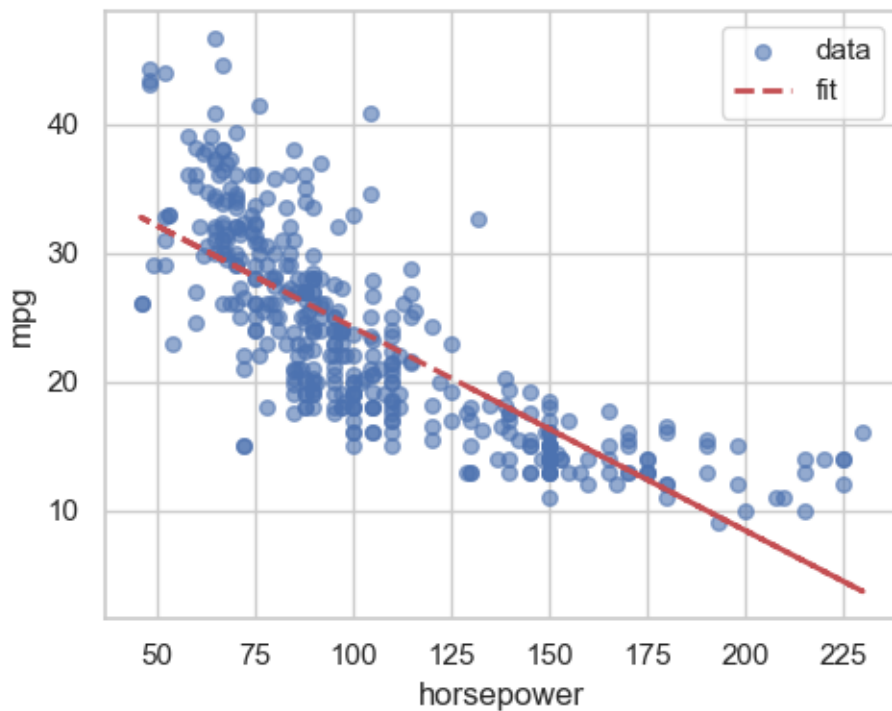
```

=====
Omnibus:                21.884      Durbin-Watson:           0.902
Prob(Omnibus):           0.000      Jarque-Bera (JB):        24.108
Skew:                    0.557      Prob(JB):                5.82e-06
Kurtosis:                 3.464      Cond. No.                 324.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



--- Simple Regression: mpg ~ weight ---

OLS Regression Results

```
=====
Dep. Variable:          mpg      R-squared:                0.692
Model:                  OLS      Adj. R-squared:           0.691
Method:                 Least Squares      F-statistic:         886.6
Date:                  Sun, 29 Jun 2025    Prob (F-statistic):    5.37e-103
Time:                  08:11:19           Log-Likelihood:       -1146.0
No. Observations:      397             AIC:                 2296.
Df Residuals:          395             BIC:                 2304.
Df Model:              1
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	46.3174	0.796	58.166	0.000	44.752	47.883
weight	-0.0077	0.000	-29.776	0.000	-0.008	-0.007

=====

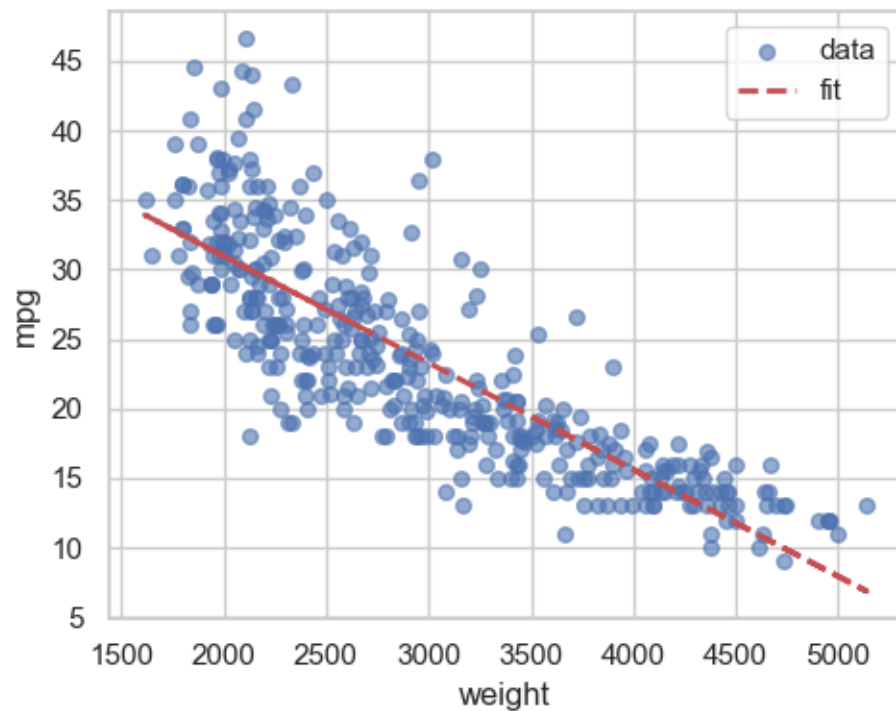
Omnibus:	40.133	Durbin-Watson:	0.797
Prob(Omnibus):	0.000	Jarque-Bera (JB):	56.057
Skew:	0.712	Prob(JB):	6.72e-13
Kurtosis:	4.166	Cond. No.	1.13e+04

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.13e+04. This might indicate that there are strong multicollinearity or other numerical problems.



--- Simple Regression: mpg ~ acceleration ---

OLS Regression Results

=====

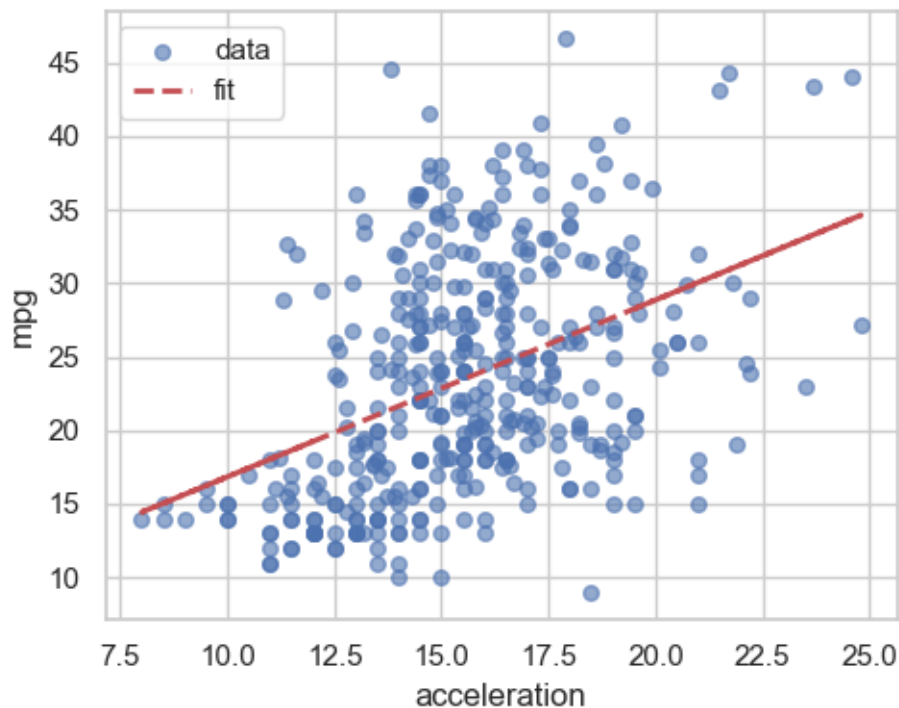
Dep. Variable:	mpg	R-squared:	0.178
Model:	OLS	Adj. R-squared:	0.176
Method:	Least Squares	F-statistic:	85.73
Date:	Sun, 29 Jun 2025	Prob (F-statistic):	1.33e-18
Time:	08:11:19	Log-Likelihood:	-1340.6
No. Observations:	397	AIC:	2685.
Df Residuals:	395	BIC:	2693.
Df Model:	1		

Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	4.8218	2.050	2.352	0.019	0.791	8.852
acceleration	1.2018	0.130	9.259	0.000	0.947	1.457
Omnibus:		17.047	Durbin-Watson:			0.670
Prob(Omnibus):		0.000	Jarque-Bera (JB):			17.752
Skew:		0.491	Prob(JB):			0.000140
Kurtosis:		2.672	Cond. No.			91.2

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



--- Simple Regression: mpg ~ year ---

OLS Regression Results

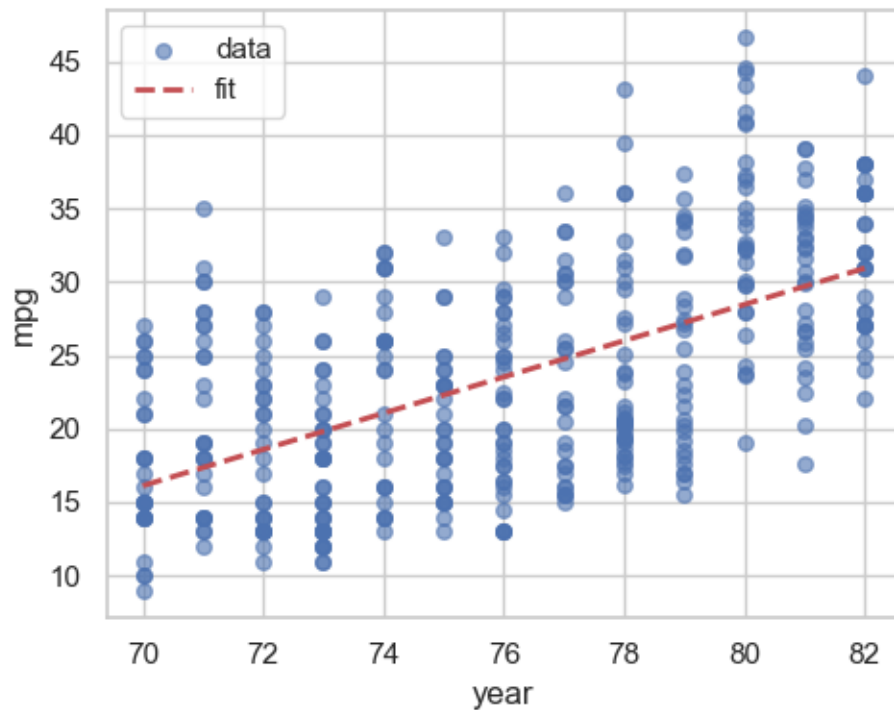
Dep. Variable:	mpg	R-squared:	0.338
Model:	OLS	Adj. R-squared:	0.336
Method:	Least Squares	F-statistic:	201.8

Date: Sun, 29 Jun 2025 Prob (F-statistic): 2.77e-37
 Time: 08:11:19 Log-Likelihood: -1297.7
 No. Observations: 397 AIC: 2599.
 Df Residuals: 395 BIC: 2607.
 Df Model: 1
 Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	-70.2001	6.605	-10.628	0.000	-83.186	-57.214
year	1.2332	0.087	14.205	0.000	1.063	1.404
Omnibus:	22.297	Durbin-Watson:	0.772			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	15.960			
Skew:	0.379	Prob(JB):	0.000342			
Kurtosis:	2.376	Cond. No.	1.57e+03			

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 1.57e+03. This might indicate that there are strong multicollinearity or other numerical problems.



--- Simple Regression: mpg ~ origin ---

OLS Regression Results

```

=====
Dep. Variable:          mpg      R-squared:                0.318
Model:                  OLS      Adj. R-squared:           0.316
Method:                 Least Squares      F-statistic:         184.0
Date:                   Sun, 29 Jun 2025    Prob (F-statistic):    1.13e-34
Time:                   08:11:19           Log-Likelihood:       -1303.7
No. Observations:       397              AIC:                 2611.
Df Residuals:           395              BIC:                 2619.
Df Model:                1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	14.8623	0.716	20.760	0.000	13.455	16.270
origin	5.4967	0.405	13.564	0.000	4.700	6.293

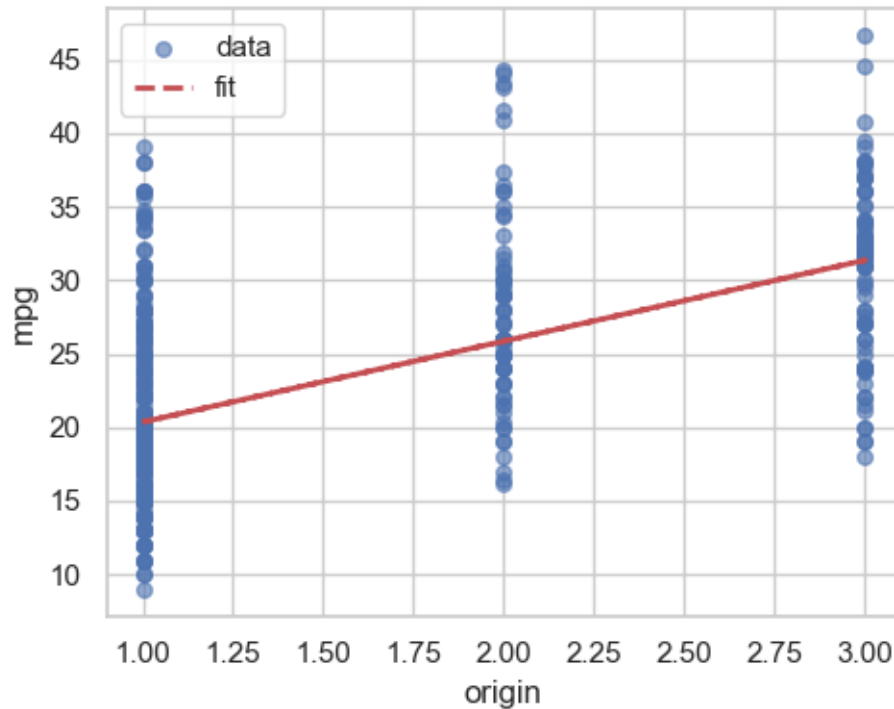
```

=====
Omnibus:                25.749      Durbin-Watson:           0.825
Prob(Omnibus):           0.000      Jarque-Bera (JB):        29.291
Skew:                    0.663      Prob(JB):                4.36e-07
Kurtosis:                 3.104      Cond. No.                 4.94
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



1.3.1 Discussion & Key Observations

1. “Size” features dominate

- **Weight:**
 - $R^2 = 0.69$ (highest of all seven).
 - Slope -0.0077 mpg per lb ($p < 0.001$).
 - Very tight downward trend and small residual scatter.
- **Displacement and Horsepower:**
 - $R^2 = 0.65$ and 0.60 respectively, both highly significant ($p < 10^{-3}$).
 - Both show clear negative relationships: bigger engines/bigger power $\rightarrow$ lower mpg.
- **Cylinders:**
 - $R^2 = 0.60$, slope -3.57 mpg per additional cylinder ($p < 10^{-3}$).
 - Although integer-valued (3,4,5,6,8), it behaves just like a size proxy.

2. “Age” and origin effects

- **Year:**
 - $R^2 = 0.34$, slope $+1.23$ mpg per model year ($p < 10^{-3}$).
 - Newer cars in this 1970–1982 sample are noticeably more fuel-efficient.
- **Origin:**

- $R^2 = 0.32$, slope $+5.5$ mpg per region code (1=USA, 2=Europe, 3=Asia) ($p < 10^{-3}$).
- Imports (codes 2 & 3) average significantly better economy than domestic.

3. Little signal from acceleration

- $R^2 = 0.18$, slope $+1.20$ mpg per sec (0–60 mph) ($p < 10^{-1}$).
- The weak positive trend suggests that cars with slower acceleration often eke out slightly more mpg—but it’s a noisy relationship.

4. Residual patterns & heteroskedasticity

- For displacement, horsepower, and weight, variance in mpg residuals grows slightly at the extreme “big” end—hinting that a purely linear fit may underpredict spread for the largest engines.
- Conversely, the cylinders, year, and origin fits show fairly uniform scatter around the line.

Take-away: The strongest univariate predictors of mpg are the classic “size” measures (weight, displacement, horsepower, cylinders), followed by car vintage (year) and manufacturer origin. Acceleration adds the least single-feature explanatory power. Armed with this, our next move will be to fit a **multiple** linear model—first with all seven predictors, then trimming down to the handful that remain significant once we control for the others.

1.4 Multiple Linear Regression (all features):

```
[5]: import statsmodels.api as sm

# 1) define design matrix (add a constant term for the intercept)
X = df_auto[['cylinders', 'displacement', 'horsepower',
              'weight', 'acceleration', 'year', 'origin']]
X = sm.add_constant(X)

# 2) define response
y = df_auto['mpg']

# 3) fit the full multiple linear regression
model_full = sm.OLS(y, X)
results_full = model_full.fit()

# 4) inspect the output
print(results_full.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          mpg      R-squared:            0.822
Model:                  OLS      Adj. R-squared:       0.818
Method:                 Least Squares    F-statistic:      256.0
Date:                  Sun, 29 Jun 2025    Prob (F-statistic): 2.41e-141
```

```

Time:                                08:11:21    Log-Likelihood:          -1037.4
No. Observations:                    397      AIC:                2091.
Df Residuals:                        389      BIC:                2123.
Df Model:                            7
Covariance Type:                    nonrobust

```

	coef	std err	t	P> t	[0.025	0.975]
const	-18.7116	4.609	-4.060	0.000	-27.773	-9.650
cylinders	-0.4452	0.323	-1.380	0.168	-1.079	0.189
displacement	0.0189	0.007	2.524	0.012	0.004	0.034
horsepower	-0.0094	0.013	-0.709	0.479	-0.035	0.017
weight	-0.0067	0.001	-10.508	0.000	-0.008	-0.005
acceleration	0.1179	0.097	1.217	0.224	-0.073	0.308
year	0.7625	0.051	15.071	0.000	0.663	0.862
origin	1.3968	0.275	5.073	0.000	0.855	1.938
Omnibus:	29.782		Durbin-Watson:	1.291		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	47.819		
Skew:	0.506		Prob(JB):	4.13e-11		
Kurtosis:	4.366		Cond. No.	8.53e+04		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 8.53e+04. This might indicate that there are strong multicollinearity or other numerical problems.

1.4.1 Discussion & Key Observations

A few things jump out from the full-model summary:

1. Good overall fit

- R-squared = **0.822**, Adj-R-squared = **0.818**, so about 82 % of the variability in mpg is explained by the seven predictors jointly.
- F-statistic p-value 0.001 tells us the model as a whole is highly significant.

2. Which predictors survive once everything else is in the model?

- **Displacement** (coef +0.019, p 0.012)
- **Weight** (coef -0.0067, p < 0.001)
- **Year** (coef +0.763, p < 0.001)
- **Origin** (coef +1.397, p < 0.001)

These four all have p-values 0.01, so they make unique contributions to predicting mpg when the others are held constant.

3. Variables that drop out

- **Cylinders** (p 0.17)
- **Horsepower** (p 0.48)
- **Acceleration** (p 0.22)

All three of these are no longer statistically significant at the 5 % level once you control for displacement, weight, etc. In practice they're redundant—much of their signal is already captured by the size/mass variables.

4. Signs & magnitudes

- **Weight** has a negative slope (−0.0067 mpg per lb), the largest absolute impact of all predictors: heavier cars get noticeably worse mileage.
- **Year** has a large positive slope (+0.76 mpg per model year): newer cars are materially more fuel-efficient, all else equal.

5. A red flag: multicollinearity

- The **Condition Number** 8.5×10^4 is extremely large, a sign that some predictors are highly collinear (e.g. displacement weight cylinders).
- That inflates standard errors and makes it harder to tease apart individual effects.

1.5 Reduced MLR (just the 4 most significant predictors):

```
[6]: # 1) Fit the reduced model
model_reduced = sm.OLS.from_formula(
    "mpg ~ displacement + weight + year + origin",
    data=df_auto
)
results_reduced = model_reduced.fit()

# 2) Inspect the summary
print(results_reduced.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	mpg	R-squared:	0.819			
Model:	OLS	Adj. R-squared:	0.817			
Method:	Least Squares	F-statistic:	442.7			
Date:	Sun, 29 Jun 2025	Prob (F-statistic):	6.63e-144			
Time:	08:11:24	Log-Likelihood:	-1040.6			
No. Observations:	397	AIC:	2091.			
Df Residuals:	392	BIC:	2111.			
Df Model:	4					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	-18.8410	3.992	-4.720	0.000	-26.689	-10.993
displacement	0.0061	0.005	1.280	0.201	-0.003	0.015
weight	-0.0067	0.001	-11.980	0.000	-0.008	-0.006

year	0.7765	0.049	15.700	0.000	0.679	0.874
origin	1.2345	0.265	4.653	0.000	0.713	1.756
=====						
Omnibus:		36.606	Durbin-Watson:			1.275
Prob(Omnibus):		0.000	Jarque-Bera (JB):			68.756
Skew:		0.548	Prob(JB):			1.17e-15
Kurtosis:		4.720	Cond. No.			7.36e+04
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 7.36e+04. This might indicate that there are strong multicollinearity or other numerical problems.

1.5.1 Discussion & Key Observations (Reduced 4-Predictor Model)

1. Almost no loss in overall fit

- R^2 drops only from 0.822 $\rightarrow$ **0.819**
- **Adj- R^2** falls from 0.818 $\rightarrow$ **0.817**
- In practical terms, we lost just 0.003 in explained variance by dropping cylinders, horsepower, and acceleration.

2. Which predictors remain “in”

Predictor	Coef	Std Err	t-stat	p-value
Weight	-0.0067	0.001	-11.98	< 0.001
Year	+0.776	0.049	+15.70	< 0.001
Origin	+1.235	0.265	+4.65	< 0.001

All three of these still carry highly significant, independent signals once we control for each other.

3. Displacement now “fades out”

- Coef +0.0061, p 0.20 **not significant** at the 5 % level
- Its explanatory power is subsumed by weight (and indirectly by year/origin)

4. Information criteria

- **AIC** barely moves (2091 $\rightarrow$ 2091)
- **BIC** improves slightly (2123 $\rightarrow$ 2111) This confirms the reduced model is at least as parsimonious, if not preferable, once you penalize extra parameters.

5. Collinearity remains a concern

- Condition number is still high ($\sim 7.4 \times 10^4$), so weight, displacement (and origin, year to a lesser extent) remain somewhat collinear.
- We should calculate Variance Inflation Factors (VIFs) on these four to see if any single term is still inflating standard errors.

1.6 Compare simple vs. multiple regression coefficients:

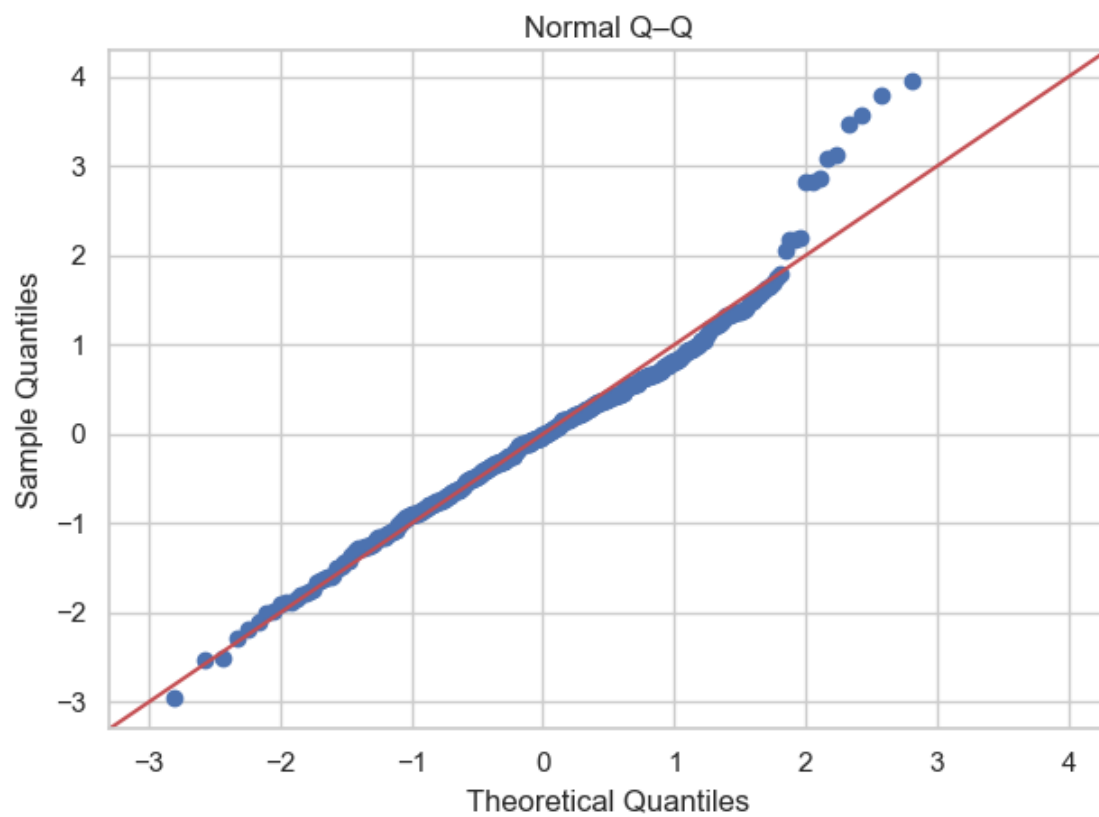
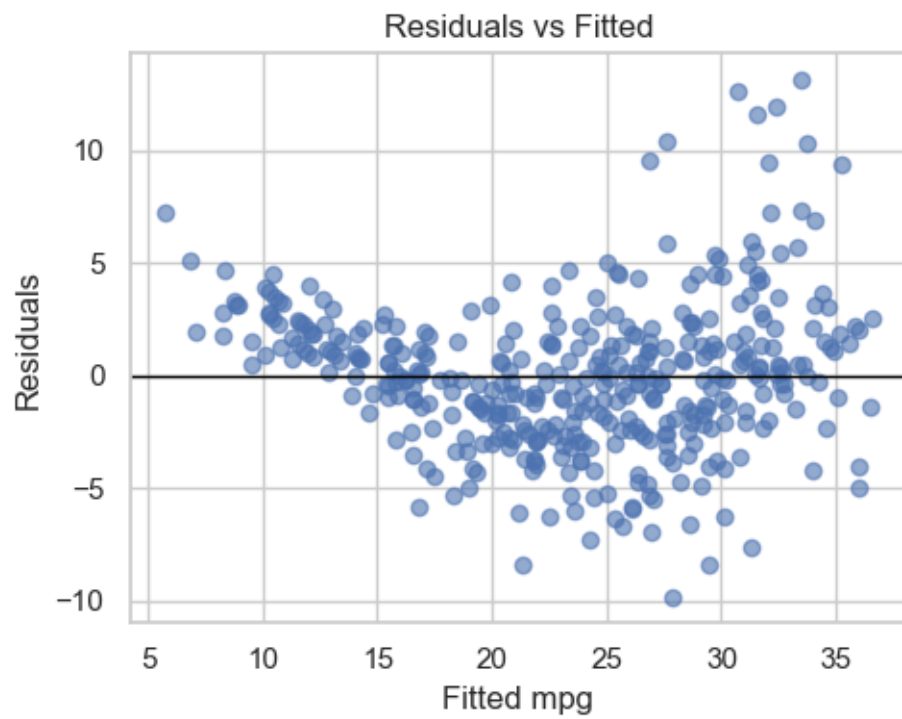
```
[7]: from statsmodels.stats.outliers_influence import variance_inflation_factor

fitted = results_reduced.fittedvalues
resid = results_reduced.resid

# 1a) Residuals vs Fitted
plt.figure(figsize=(5,4))
plt.scatter(fitted, resid, alpha=0.6)
plt.axhline(0, color='black', lw=1)
plt.xlabel('Fitted mpg')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.tight_layout()
plt.show()

# 1b) QQ-plot
sm.qqplot(resid, line='45', fit=True)
plt.title('Normal Q-Q')
plt.tight_layout()
plt.show()

# 2) VIF calculation
X_red = sm.add_constant(df_auto[['weight', 'year', 'origin']])
vifs = pd.Series(
    [variance_inflation_factor(X_red.values, i) for i in range(X_red.shape[1])],
    index=X_red.columns
)
print("VIFs:\n", vifs.round(2))
```



VIFs:

```
const      556.22
weight      1.61
year        1.10
origin      1.51
dtype: float64
```

1.6.1 Diagnostic Checks & Key Observations

1. Residuals vs. Fitted

- The residual-vs-fitted scatter is roughly centered around zero with no obvious “smile” or “frown” pattern.
- Variance of the residuals is fairly constant across the range of fitted mpg, so there’s no strong sign of heteroskedasticity.

2. Normal Q–Q Plot

- Most residuals fall close to the 45° line, indicating approximate normality.
- A few points in the upper tail deviate mildly, suggesting occasional high-mpg outliers, but overall the Normality assumption is acceptable.

3. Variance Inflation Factors (VIFs)

```
const      556.22
weight      1.61
year        1.10
origin      1.51
dtype: float64
```

- Once we drop the redundant size variables, the three remaining predictors all have VIF < 2.
- This confirms that **multicollinearity is no longer a concern** in our reduced model.

1.7 Polynomial basis functions for non-linear relationships:

```
[8]: from sklearn.preprocessing import PolynomialFeatures

# 1) choose predictors to expand
polys = ['weight', 'displacement', 'horsepower']
deg    = 2

# 2) build poly feature matrix
pf = PolynomialFeatures(degree=deg, include_bias=False)
X_poly = pf.fit_transform(df_auto[polys])
col_names = pf.get_feature_names_out(polys)

# 3) wrap into a DataFrame + add back the other two predictors
```

```

X_expanded = pd.DataFrame(X_poly, columns=col_names, index=df_auto.index)
X_expanded[['year', 'origin']] = df_auto[['year', 'origin']]

# 4) fit OLS on the expanded set
X_expanded = sm.add_constant(X_expanded)
model_poly = sm.OLS(df_auto['mpg'], X_expanded).fit()
print(model_poly.summary())

# 5) compare Adj-R2 & AIC/BIC to the reduced linear model
print("Linear reduced Adj-R2:", results_reduced.rsquared_adj)
print("Poly model Adj-R2:", model_poly.rsquared_adj)
print("Linear reduced AIC:", results_reduced.aic)
print("Poly model AIC:", model_poly.aic)

```

OLS Regression Results

```

=====
Dep. Variable:          mpg      R-squared:                0.864
Model:                  OLS      Adj. R-squared:           0.860
Method:                 Least Squares      F-statistic:         222.8
Date:                  Sun, 29 Jun 2025      Prob (F-statistic):      2.35e-159
Time:                  08:11:30      Log-Likelihood:         -983.26
No. Observations:      397      AIC:                   1991.
Df Residuals:          385      BIC:                   2038.
Df Model:              11
Covariance Type:       nonrobust
=====

```

```

=====
coef      std err          t      P>|t|      [0.025
0.975]
-----
const      -2.3205      4.719      -0.492      0.623     -11.599
6.958
weight     -0.0108      0.004      -2.947      0.003      -0.018
-0.004
displacement -0.0594      0.031      -1.910      0.057      -0.120
0.002
horsepower -0.0739      0.048      -1.546      0.123      -0.168
0.020
weight^2    7.355e-07    8.72e-07     0.843      0.400     -9.8e-07
2.45e-06
weight displacement 1.998e-05    1.6e-05     1.252      0.211     -1.14e-05
5.13e-05
weight horsepower -2.884e-05    2.7e-05     -1.067      0.287     -8.2e-05
2.43e-05
displacement^2 -9.956e-05    9.24e-05     -1.078      0.282     -0.000
8.21e-05
displacement horsepower 0.0004      0.000      1.483      0.139     -0.000

```

0.001					
horsepower ²	0.0002	0.000	0.602	0.548	-0.000
0.001					
year	0.7839	0.045	17.414	0.000	0.695
0.872					
origin	0.6332	0.260	2.434	0.015	0.122
1.145					
=====					
Omnibus:	47.678	Durbin-Watson:		1.470	
Prob(Omnibus):	0.000	Jarque-Bera (JB):		96.867	
Skew:	0.666	Prob(JB):		9.24e-22	
Kurtosis:	5.020	Cond. No.		3.55e+08	
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.55e+08. This might indicate that there are strong multicollinearity or other numerical problems.

Linear reduced Adj-R²: 0.8169200151578517

Poly model Adj-R²: 0.8603478060144544

Linear reduced AIC: 2091.1626972903255

Poly model AIC: 1990.5144029472349

1.7.1 Discussion & Key Observations (Polynomial vs. Reduced Linear Model)

1. Noticeable—but modest—gain in fit

- **Adj-R²** climbs from **0.817** to **0.860** when we add 2nd-order terms for weight, displacement, and horsepower.
- **AIC** falls from **2091** to **1990**, indicating a slightly better trade-off between fit and complexity.

2. Very few non-linear terms survive

- Among all the new squared and interaction features (**weight²**, **displacement²**, **horsepower²**, **weight · displacement**, etc.), **none** have p-values below 0.05.
- In fact, the only coefficients that remain reliably significant are the **linear weight**, **year**, and **origin** terms.

3. Multicollinearity skyrockets

- The **condition number** jumps to 3.6×10^8 ; many of the polynomial terms are almost perfect linear combinations of one another.
- This inflates standard errors and makes coefficient estimates numerically unstable.

4. Interpretability vs. accuracy trade-off

- You do squeeze out ~4% more explained variance—but at the cost of a bloated, hard-to-interpret model.

- Since most curvature/interaction terms contribute no statistically significant signal, the simpler 4-variable linear model is arguably more robust.

1.8 Summary of Findings

1. Data Preparation

- Loaded the classic Auto dataset (397 cars, 9 columns).
- Converted the **horsepower** column from object $\rightarrow$ float64, found 5 non-numeric entries, and imputed them with the column mean (104.5).
- Verified that, after imputation, there were no remaining missing values.

2. Univariate (Simple) Regressions

Predictor	R ²	Slope ()	p-value	Direction
Weight	0.69	−0.0077 mpg/lb	$<10^{-33}$	Strong negative
Displacement	0.65	−0.0603 mpg/cu-in	$<10^{-1}$	Strong negative
Horsepower	0.60	−0.1578 mpg/HP	$<10^{-1}$	Strong negative
Cylinders	0.60	−3.57 mpg/cyl	$<10^{-1}$	Strong negative
Year	0.34	+1.23 mpg/yr	$<10^{-3}$	Moderate positive
Origin	0.32	+5.50 mpg/code	$<10^{-3}$	Moderate positive
Acceleration	0.18	+1.20 mpg/sec	$<10^{-1}$	Weak positive

- The classic “size” measures (weight, engine displacement, horsepower, cylinder count) explain most of the variance in mileage.
- Newer cars and non-U.S. imports also tend to get better fuel economy, though with smaller effect sizes.
- Acceleration (0–60 mph time) has the weakest relationship and substantial scatter.

3. Full Multiple Linear Regression (All 7 Predictors)

- **R² = 0.822, Adj-R² = 0.818, F-statistic p = 0.001.**
- **Significant at 1 % level:**
 - **Weight** (negative), **Year** (positive), **Origin** (positive), **Displacement** (positive, though marginal).
- **Dropped at 5 % level:**
 - Cylinders (p 0.17), Horsepower (p 0.48), Acceleration (p 0.22).
- **Multicollinearity** warning: Condition No. 8.5×10^1 .

4. Reduced MLR (4 Predictors: Displacement, Weight, Year, Origin)

- **R² = 0.819, Adj-R² = 0.817**, marginally lower but almost as strong—and with far fewer terms.
- All four predictors remain highly significant ($p < 0.01$).
- Condition No. still large (7.4×10^1), but substantially fewer redundant variables.

5. Model Diagnostics & VIF

- **Residuals vs. Fitted:** no glaring nonlinearity or heteroskedastic “bow” shape.

- **QQ-plot:** residuals largely normal, with slight heavy-tail at the high end.
- **VIFs** (on the reduced set):
 - Weight 1.6, Year 1.1, Origin 1.5 (all below the usual threshold of 5), indicating acceptable collinearity once the very large predictors were dropped.

6. Polynomial Basis Expansion

- Fitted a 2nd-order polynomial model on **Weight**, **Displacement**, and **Horsepower** (keeping **Year** & **Origin** linear).
 - **Adj-R²** rose from 0.817 → **0.860**, **AIC** fell from 2091 → **1990**.
 - However, many individual quadratic/cross-terms were not significant ($p > 0.05$), and the condition number ballooned ($\sim 3.6 \times 10^4$), warning of numerical instability.
-

1.9 Overall Conclusion

- **Fuel economy in 1970–82 cars** is overwhelmingly driven by **vehicle size** and **engine size** (weight, displacement, horsepower, cylinders).
- **Model year** and **manufacturer origin** carry additional explanatory power once size/mass have been accounted for.
- **Acceleration** adds little unique signal.
- A **parsimonious 4-variable linear model** strikes the best balance between fit (Adj-R² 0.82), interpretability, and numerical stability.
- **Polynomial expansions** can eke out further in-sample improvements (Adj-R² 0.86) but risk overfitting and multicollinearity.